Name : KEERTHANA R

Reg No : 411625104031
Department : CSE
Subject : JAVA (OOPS)
Batch : 4
Title : Chocolate Program Demonstrating Compile-time and Run-time Polymorphism

Aim

To design and implement a Java program that demonstrates both compile-time polymorphism (method overloading) and run-time polymorphism (method overriding) using a chocolate example.

Procedure

Step 1: Creating Base Class

- A class named Chocolate was created.
- It contains overloaded methods showDetails() to demonstrate compile-time polymorphism.
- It also has a displayInfo() method to be overridden by subclasses.

Step 2: Implementing Subclasses

- A subclass Brownie extends Chocolate and overrides displayInfo() to show brownie-specific details.
- Another subclass IceCream extends Chocolate and overrides displayInfo() to show ice-cream-specific details.

Step 3: Designing the Main Class

- The Main class contains the main() method.
- Objects of Brownie and IceCream are created but referenced using the parent class Chocolate.

- This demonstrates run-time polymorphism.
- The overloaded showDetails() methods are invoked to demonstrate compile-time polymorphism.

Step 4: Applying Object-Oriented Concepts

- **Inheritance** – Brownie and IceCream extend Chocolate.
- **Polymorphism** –
    - Compile-time: Method overloading (showDetails).
    - Run-time: Method overriding (displayInfo).
- **Encapsulation** – Attributes like ingredients and pieces are maintained inside their respective classes.

Step 5: Executing the Program

- The program creates chocolate objects and calls both overridden and overloaded methods.
- Output is displayed based on the type of chocolate and the method invoked.

Step 6: Testing

- Tested with different chocolate types (Brownie, IceCream).
- Verified correct outputs for overridden methods.
- Verified compile-time polymorphism by calling overloaded methods with different argument types.

Code

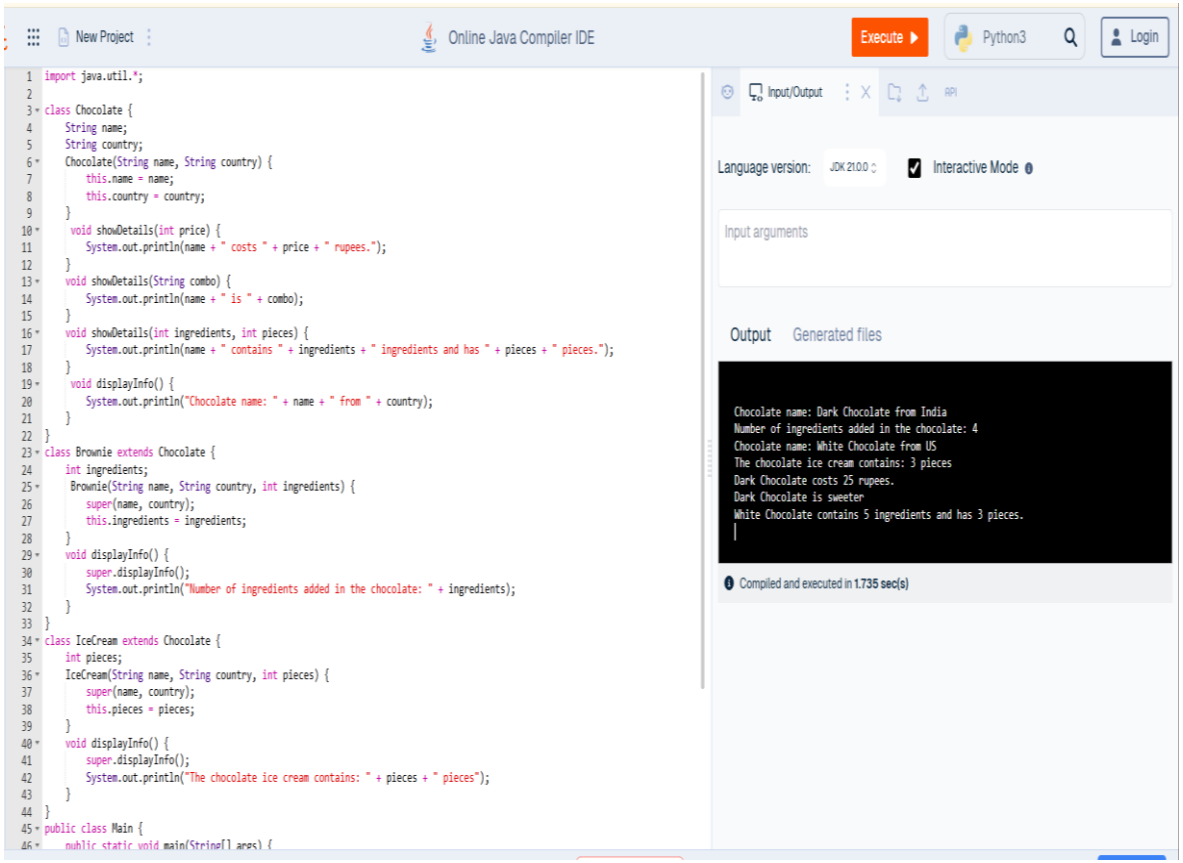
```
import java.util.*;


class Chocolate {
```

```
    String name;
    String country;
    Chocolate(String name, String country) {
        this.name = name;
        this.country = country;
    }
     void showDetails(int price) {
        System.out.println(name + " costs " + price + " rupees.");
    }
    void showDetails(String combo) {
        System.out.println(name + " is " + combo);
    }
    void showDetails(int ingredients, int pieces) {
        System.out.println(name + " contains " + ingredients + " ingredients and has " + pieces + " pieces.");
    }
     void displayInfo() {
        System.out.println("Chocolate name: " + name + " from " + country);
    }
}
```

```
class Brownie extends Chocolate {
    int ingredients;
     Brownie(String name, String country, int ingredients) {
        super(name, country);
        this.ingredients = ingredients;
    }
    void displayInfo() {
        super.displayInfo();
        System.out.println("Number of ingredients added in the chocolate: " + ingredients);
    }
}
class IceCream extends Chocolate {
    int pieces;
    IceCream(String name, String country, int pieces) {
        super(name, country);
        this.pieces = pieces;
    }
    void displayInfo() {
        super.displayInfo();
```

```
        System.out.println("The chocolate ice cream contains: " + pieces + " pieces");
    }
}
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Chocolate c1 = new Brownie("Dark Chocolate", "India", 4);
        Chocolate c2 = new IceCream("White Chocolate", "US", 3);
        c1.displayInfo();
        c2.displayInfo();
        c1.showDetails(25);
        c1.showDetails("sweeter");
        c2.showDetails(5, 3);
         sc.close();
    }
}
```




```java
import java.util.*;

class Chocolate {
    String name;
    String country;
    Chocolate(String name, String country) {
        this.name = name;
        this.country = country;
    }
     void showDetails(int price) {
        System.out.println(name + " costs " + price + " rupees.");
    }
    void showDetails(String combo) {
        System.out.println(name + " is " + combo);
    }
    void showDetails(int ingredients, int pieces) {
        System.out.println(name + " contains " + ingredients + " ingredients and has " + pieces + " pieces.");
    }
     void displayInfo() {
        System.out.println("Chocolate name: " + name + " from " + country);
    }
}
class Brownie extends Chocolate {
    int ingredients;
     Brownie(String name, String country, int ingredients) {
        super(name, country);
        this.ingredients = ingredients;
    }
    void displayInfo() {
        super.displayInfo();
        System.out.println("Number of ingredients added in the chocolate: " + ingredients);
    }
}
class IceCream extends Chocolate {
    int pieces;
    IceCream(String name, String country, int pieces) {
        super(name, country);
        this.pieces = pieces;
    }
    void displayInfo() {
        super.displayInfo();
        System.out.println("The chocolate ice cream contains: " + pieces + " pieces");
    }
}
public class Main {
    public static void main(String[] args) {
```

Result

The program was successfully executed. It clearly demonstrates **compile-time polymorphism** through method overloading and **run-time polymorphism** through method overriding, using a chocolate example.

Would you like me to also prepare a **sample output screenshot (text output)** section, so your lab record looks complete with expected results?